ViewRendererDialog コンポーネント UI仕様書

1. 概要

1.1 コンポーネントの目的

ViewRendererDialogは、ファイルまたはサブネットをフローティングダイアログで表示するコンポーネントです。react-rndライブラリを使用してドラッグ&リサイズが可能で、複数のダイアログを重ねて表示できます。

1.2 使用場面

- プロジェクト管理画面でファイルを新しいウィンドウで開く際
- Petri Netビューアーでサブネットを別ウィンドウで表示する際
- 複数のファイルを同時に参照する必要がある場合
- ファイルを編集モードで開く際（editモード）

1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造

2.1.1 通常表示時

```
┌──────────────────────────────────────────────────────┐
│ [ファイル名]                    [最小化] [閉じる] ← タイトルバー │
├──────────────────────────────────────────────────────┤
│                                                      │
```

```
│  <ViewRenderer /> または <SubNetViewer />              │
│  (コンテンツエリア)                                    │
│                                                        │
│                                                        │
└────────────────────────────────────────────────────────┘
```

2.1.2 最小化時

```
┌────────────────────────────────────────────────────────┐
│ [ファイル名]                    [展開] [閉じる]        │
└────────────────────────────────────────────────────────┘
```

2.1.3 ロード中

```
┌────────────────────────────────────────────────────────┐
│ [ファイル名]                    [最小化] [閉じる]      │
├────────────────────────────────────────────────────────┤
│                                                        │
│                  🔄 Spinner                            │
│                  ロード中...                           │
│                                                        │
└────────────────────────────────────────────────────────┘
```

2.2 主要UI要素

2.2.1 タイトルバー

- **背景色**: #fafafa（グレー背景）
- **ドラッグハンドル**: タイトルバー全体（カーソルが `move` に変化）
- **タイトル**: ファイル名またはサブネット名（太字、16px）
- **最小化ボタン**: MinusOutlined/ExpandOutlined アイコン
- **閉じるボタン**: CloseOutlined アイコン

2.2.2 コンテンツエリア

- **背景色**: #fff（白背景）
- **境界**: 角丸（8px）、シャドウ付き

- **内容**: ViewRendererまたはSubNetViewer
- **スクロール**: overflow: hidden（子コンポーネントで制御）

2.2.3 ダイアログ配置

- **初期位置**: x: 100, y: 100（画面左上からのオフセット）
- **初期サイズ**: 1200x800（幅x高さ）
- **最小サイズ**:
  - subnetモード: 1000x600
  - その他: 800x600
- **最大サイズ**: 画面サイズ - 100px
- **境界**: window内に制限

---

3. 機能仕様

3.1 ファイルコンテンツの読み込み

3.1.1 処理フロー

```
useEffect(() => {
  if ((contentType !== 'file' && contentType !== 'edit') || !file) {
    return;
  }

  // 外部からコンテンツが渡されている場合はそのまま使用（APIスキップ）
  if (externalFileContent !== undefined) {
    setFileContent(externalFileContent ?? null);
    return;
  }

  // MAIMLファイルのみAPIからテキストを取得
  if (file.type !== 'maiml') {
    setFileContent(null);
    return;
  }
```

```
  let isMounted = true;

  const loadContent = async () => {
    setLoading(true);
    try {
      console.log('🔍 [ViewRendererDialog] MAIMLファイルをS3から読み込み中:', file.name,
'path:', file.path);
      const content = await getFileContentAsText(file.path);
      if (isMounted) {
        setFileContent(content);
        console.log('✅ [ViewRendererDialog] ファイル読み込み成功:', { size: content.length });
      }
    } catch (error) {
      console.error('❌ [ViewRendererDialog] ファイル読み込みエラー:', error);
      message.error(`ファイルの読み込みに失敗しました: ${error instanceof Error ? error.message :
'不明なエラー'}`);
      if (isMounted) {
        setFileContent(null);
      }
    } finally {
      if (isMounted) {
        setLoading(false);
      }
    }
  };

  loadContent();

  return () => {
    isMounted = false;
  };
}, [contentType, file, externalFileContent]);
```

3.1.2 UI動作

- **開始時**: loadingフラグをtrueに設定、Spinner表示
- **成功時**: fileContentをStateに設定、ViewRendererに渡す
- **エラー時**: message.errorでエラーメッセージ表示、fileContentはnull
- **完了時**: loadingフラグをfalseに設定

3.1.3 外部コンテンツ対応

- `externalFileContent` Propsが渡された場合はAPIスキップ

- 親コンポーネントが既にファイル内容を持っている場合に使用

3.2 ダイアログの操作

3.2.1 ドラッグ移動

- **対象**: タイトルバー（`drag-handle`クラス）
- **範囲**: window内に制限（`bounds="window"`）
- **処理**: react-rndが自動処理

3.2.2 リサイズ

- **有効条件**: 最小化されていない時のみ
- **最小サイズ**: contentTypeによって異なる（800x600 または 1000x600）
- **最大サイズ**: 画面サイズ - 100px
- **処理**: react-rndが自動処理

3.2.3 最小化/展開

```
<Button
  type="text"
  icon={isMinimized ? <ExpandOutlined /> : <MinusOutlined />}
  onClick={(e) => {
    e.stopPropagation();
    setIsMinimized(!isMinimized);
  }}
  title={isMinimized ? '展開' : '最小化'}
/>
```

- **最小化時**: 高さ60pxに縮小、コンテンツ非表示、リサイズ無効
- **展開時**: 元のサイズに戻る、コンテンツ表示、リサイズ有効

3.2.4 閉じる

```
<Button
  type="text"
  icon={<CloseOutlined />}
  onClick={(e) => {
    e.stopPropagation();
    onClose(id);
  }}
  title="閉じる"
/>
```

- **処理**: 親コンポーネントの `onClose` コールバックを呼び出し

3.2.5 フォーカス

```
onMouseDown={() => onFocus(id)}
```

- **タイミング**: ダイアログ内をクリックした時
- **処理**: 親コンポーネントの `onFocus` コールバックを呼び出し、z-indexを更新

3.3 コンテンツタイプ別レンダリング

3.3.1 fileモード

```
<ViewRenderer
  currentView="main"
  selectedFile={selectedFile}
  fileContent={fileContent}
  enhancedMode={false}
  downloadingFile={false}
  deviceType="desktop"
  isTabletDevice={false}
  isMobileDevice={false}
  onOpenSubNetViewer={onOpenSubNetViewer}
/>
```

- **用途**: 通常のファイルビューアー表示

- **対応ファイル**: MAIML, TIFF, 画像ファイル

3.3.2 editモード

```
<ViewRenderer
  currentView="edit"
  selectedFile={selectedFile}
  fileContent={fileContent}
  enhancedMode={false}
  downloadingFile={false}
  deviceType="desktop"
  isTabletDevice={false}
  isMobileDevice={false}
  onExitEdit={() => onClose(id)}
  onOpenSubNetViewer={onOpenSubNetViewer}
/>
```

- **用途**: Petri Net編集エディタ表示
- **タイトル**: "🚀 編集: [ファイル名]"
- **終了時**: ダイアログを閉じる

3.3.3 subnetモード

```
<SubNetViewer
  open={true}
  nodeName={subnetData.transitionName}
  maimlData={subnetData.maimlData}
  templateMap={subnetData.templateMap}
  petriNetData={subnetData.petriNetData}
  onClose={() => onClose(id)}
/>
```

- **用途**: サブネットの詳細表示
- **最小幅**: 1000px（他より大きい）

3.4 Portal レンダリング

```
if (typeof document !== 'undefined') {
  return ReactDOM.createPortal(dialogContent, document.body);
}
return null;
```

- **目的**: ダイアログをdocument.bodyに直接レンダリング（z-index制御のため）
- **SSR対応**: document未定義時はnullを返す

---

4. データ仕様

4.1 Props定義

```
/**
 * ファイルデータ型定義
 */
export interface FileData {
  id: string;
  name: string;
  path: string;
  size: number;
  lastModified: string;
  type: string;
  isFolder?: boolean;
  fileCount?: number;
}

/**
 * サブネットデータ型定義
 */
export interface SubnetData {
  transitionName: string;
  maimlData: string;
  templateMap: Record<string, { element: Element }>;
  petriNetData: {
    places: Array<{
      id: string;
      data: {
        label?: string;
        stereotype?: string;
        template?: { id: string };
```

```
        templateId?: string;
      };
    }>;
    arcs: Array<{
      id: string;
      source: string;
      target: string;
    }>;
  };
}

/**
 * ViewRendererDialog Props
 */
interface ViewRendererDialogProps {
  id: string;
  contentType: 'file' | 'subnet' | 'edit';
  file?: FileData;
  /** 外部から渡すファイルコンテンツ（省略時はAPIから取得） */
  fileContent?: string | null;
  subnetData?: SubnetData;
  zIndex: number;
  initialPosition?: { x: number; y: number };
  initialSize?: { width: number; height: number };
  onClose: (id: string) => void;
  onFocus: (id: string) => void;
  onOpenSubNetViewer?: (subnetData: SubnetData) => void;
}
```

Props説明:

Prop名	型	必須	説明
id	string	✅	ダイアログの一意識別子
contentType	'file' | 'subnet' | 'edit'	✅	コンテンツタイプ（file/subnet/edit）
file	FileData	-	ファイル情報（fileモード、editモード時）
fileContent	string | null	-	外部から渡すファイルコンテンツ（省略時はAPI取得）
subnetData	SubnetData	-	サブネットデータ（subnetモード時）
zIndex	number	✅	ダイアログの重ね順
initialPosition	{ x: number; y: number }	-	初期表示位置（デフォルト: {x:100, y:100}）
initialSize	{ width: number; height: number }	-	初期サイズ（デフォルト: {width:1200, height:800}）

onClose	Function	✅	ダイアログを閉じる時のコールバック
onFocus	Function	✅	ダイアログがフォーカスされた時のコールバック
onOpenSubNetViewer	Function	-	サブネットビューアーを開く時のコールバック

4.2 State定義

```
const [fileContent, setFileContent] = useState<string | null>(null);
const [loading, setLoading] = useState(false);
const [isMinimized, setIsMinimized] = useState(false);
```

State説明:

State名	型	初期値	説明
fileContent	string | null	null	読み込まれたファイルの内容
loading	boolean	false	ファイルロード中フラグ
isMinimized	boolean	false	最小化状態フラグ

4.3 その他の型定義

タイトル生成ロジック:

```
const getTitle = () => {
  if (contentType === 'file' && file) {
    return file.name;
  }
  if (contentType === 'subnet' && subnetData) {
    return `サブネット: ${subnetData.transitionName}`;
  }
  if (contentType === 'edit' && file) {
    return `🚀 編集: ${file.name}`;
  }
  return '不明';
};
```

---

5. API連携

5.1 使用するAPI一覧

#	API名	メソッド	エンドポイント	状態	説明
1	getFileContentAsText	-	-	✅ 実装済み	S3からファイル内容を取得（fileAccessService）

5.2 API呼び出しコード

```
const content = await getFileContentAsText(file.path);
```

備考:

- `getFileContentAsText` は `fileAccessService` で定義されたサービス関数
- S3から署名付きURLを取得してファイル内容を読み込む
- MAIMLファイル（type='maiml'）の場合のみ呼び出される
- 外部から `fileContent` Propsが渡された場合はAPIスキップ

---

6. エラーハンドリング

6.1 エラーケース一覧

エラーケース	HTTPステータス	対処方法
ファイル読み込み失敗	-	message.error表示、コンソールエラー出力
APIエラー	400/500	message.error表示、fileContentをnull設定

6.2 エラー処理コード

```
try {
  const content = await getFileContentAsText(file.path);
  if (isMounted) {
    setFileContent(content);
  }
} catch (error) {
  console.error('❌ [ViewRendererDialog] ファイル読み込みエラー:', error);
  message.error(`ファイルの読み込みに失敗しました: ${error instanceof Error ? error.message : '不明なエラー'}`);
  if (isMounted) {
    setFileContent(null);
  }
} finally {
  if (isMounted) {
    setLoading(false);
  }
}
```

エラーメッセージ:

- "ファイルの読み込みに失敗しました: [エラーメッセージ]"

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/viewer/ViewRendererDialog.tsx`

7.2 関連ファイル

- **サービス**: `frontend/src/services/fileAccessService.ts`（getFileContentAsText）
- **子コンポーネント**:
  - `frontend/src/components/viewer/ViewRenderer.tsx`

- `frontend/src/components/viewer/dialogs/SubNetViewer.tsx`

7.3 依存コンポーネント

- `<ViewRenderer />` - ファイルビューアー統括コンポーネント
- `<SubNetViewer />` - サブネットビューアーコンポーネント
- `<Rnd />` - react-rndライブラリのドラッグ&リサイズコンポーネント

7.4 依存ライブラリ

- `antd` : Spin, message, Button, Typography
- `@ant-design/icons` : CloseOutlined, MinusOutlined, ExpandOutlined
- `react-rnd` : Rnd
- `react-dom` : ReactDOM.createPortal

---

8. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を再作成	Cline